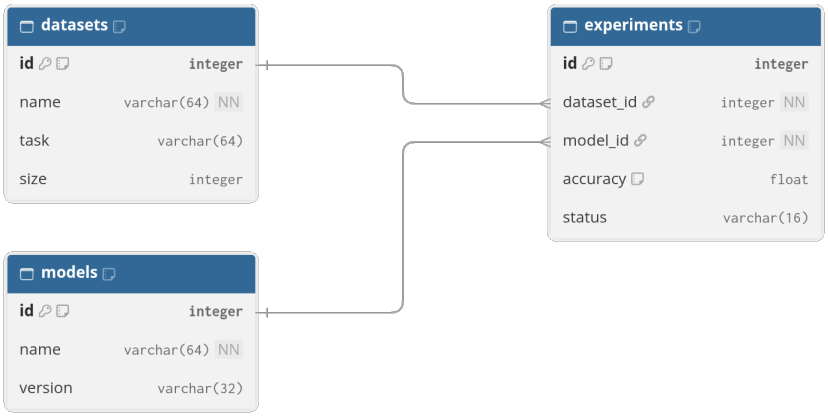
MiniDB · лабораторная работа №1

Схема, устройство и формат хранения

Мини-СУБД реализована на Rust без внешних Rust-зависимостей и готовых СУБД. JSON не используется как основное хранилище. SQL-консоль создаёт, ищет, изменяет и удаляет строки; данные сохраняются после перезапуска.

Три связанные таблицы



datasets(id, name, task, size); models(id, name, version); experiments(id, dataset_id, model_id, accuracy, status). PK — id. FK experiments ссылаются на PK datasets/models. Проверяются уникальность, авто-ID, наличие родителя и RESTRICT.

Собственные файлы

На таблицу: имя_schema — CREATE TABLE, VERSION, GENERATION и счётчик auto-ID; имя_N — единственные текущие данные, N берётся из GENERATION. UTF-8: заголовок, состояния LIVE/DEAD/FREE, значения через Tab. Строки заключены в кавычки, специальные символы экранированы. INT=i64, FLOAT=f64, STRING(n)=UTF-8 до n байт.

ALTER (ADD/DROP COLUMN, снятие FK) и VACUUM собирают живые строки в новом поколении: sync данных → атомарная замена схемы → sync каталога → удаление прежнего файла. Открытие убирает осиротевшие поколения. ALTER увеличивает VERSION и GENERATION; VACUUM — только GENERATION, если есть мёртвые слоты. Максимум auto-ID сохраняется.

Архитектура

sql.rs → Database (PK/FK, запросы, backup) → Table → TextFile. Описатель разделён на Schema (поля и ограничения) и Layout (NULL-bitmap, ширины, смещения в RAM). Row заимствует оба описателя и байты слота. UTF-8 декодируется в RAM; обычный CRUD сохраняет атомарный снимок текущего файла. Индекс PK → offset адресует слот в RAM, не байт текстового файла.

ADD/DROP перекодируют все живые строки одной таблицы: оставшиеся поля сопоставляются по имени, новые получают NULL. Ядро не пересобирается. Время зависит от старых слотов, новых байтов и числа колонок; одновременно в RAM лежат старый и новый буферы. Например, 100 тыс. строк по 100 байт + STRING(128): около 33 МБ двух буферов, без индекса. На диске до переключения тоже два поколения.

MiniDB · лабораторная работа №1

Сценарий защиты, проверки и результат индекса

Локально: ./sample.sh scratch/defense, затем ./run.sh scratch/defense. Docker: docker compose build; docker compose run --rm minidb sh /usr/local/bin/sample.sh /data/defense; docker compose run --rm minidb minidb /data/defense. Пример содержит 5 datasets, 5 models и 20 experiments; первые три experiments — из ТЗ.

Операции и ожидаемый результат

```
INSERT INTO datasets (name, task, size) VALUES ('demo', 'classification', 43);
SELECT id, name FROM datasets WHERE size >= 100;
UPDATE datasets SET size = 44 WHERE name = 'demo';
DELETE FROM datasets WHERE name = 'demo';
INSERT INTO experiments VALUES (NULL, 999, 1, 0.9, 'running');
```

INSERT выдаёт авто-ID; UPDATE и DELETE меняют по одной строке. Последний запрос отклоняется из-за FK. После EXIT и повторного открытия остаются 20 experiments. Поддерживаются шесть сравнений, AND/OR, NOT, IS NULL, INNER JOIN и backup/restore.

ADD nullable-колонки с REFERENCES сразу включает FK-проверку для следующих INSERT/UPDATE; старые строки получают NULL. DROP FOREIGN KEY сохраняет значения и снимает связь; DROP COLUMN удаляет поле и его FK. Удалять PK запрещено. Ссылки блокируют DELETE/смену PK родителя (RESTRICT). Перезапуск после ALTER не нужен; внешние правки файлов требуют повторного открытия.

Сравнение поиска с индексом и без

Release: 5000 строк, 1000 одинаковых запросов. Наполнение, открытие и sync вне таймера. Данные уже в RAM; это не измерение дискового I/O. Контрольные суммы всех трёх способов совпали: 2486500.

Просмотр — O(числа слотов); HashMap по равенству INT PK — в среднем O(1), затем проверяется весь WHERE. Диапазоны и другие поля требуют просмотра. Индекс строится при открытии и обновляется при CRUD; после переписи перестраивается.

Полный просмотр	66.159 мс
Индекс get(id)	0.143 мс
SELECT WHERE id = …	0.134 мс

Повторить: cargo run --locked --offline --release --example bench -- scratch/bench 5000. Это один локальный замер; результат зависит от машины.

Проверки и ограничения

101 автотест: CRUD, перезапуск, типы, PK/FK, JOIN, фильтры, поколения и генератор. scripts/check-docker.sh проверяет все команды в отдельном Docker-томе. Два workflow через act экспортируют drawio в README и справку --help в docs/COMMANDS.md.

Генератор читает *_schema; Rust-структуры компилирует потребитель. WAL и транзакций на несколько команд нет; пакет UPDATE ограничен 16 МиБ. Полного аудита FK сохранённых строк при открытии нет. Восстановление поколений относится к полной переписи; физическое отключение питания не проверялось.